Mastering @defer in Angular

A complete guide to lazy loading and performance optimization

Improve bundle size, Core Web Vitals & user experience

What is @defer?

The **@defer** block in Angular reduces initial bundle size by lazy loading heavy or non-critical components only when needed. This improves page load speed and Core Web Vitals like LCP & TTFB.

```
@defer {
<large-component />
}
```

Deferring with Triggers

Use different triggers to control when content loads: **idle** (default), **viewport**, **interaction**, **hover**, **immediate**, **timer**, or any custom **when** condition.

```
@defer (on viewport) {
<promo-banner />
} @placeholder {
<div>Waiting for banner...</div>
}
```

🎨 Progressive Loading (UX Blocks)

Improve UX by showing placeholders before loading, loaders while fetching, and fallback UI on failure. Dependencies in @placeholder, @loading, and @error are eagerly loaded; only the main block is deferred.

```
@defer {
<dashboard-panel />
} @placeholder {
<span>Panel will load soon...</span>
} @loading {
<span>Loading data...</span>
} @error {
<span>Error loading panel.</span>
}
```

🎯 @defer Best Practices

- Use with standalone components for true deferral.
- Avoid above-the-fold UI in defer blocks to prevent CLS.
- Assign different triggers for nested @defer blocks.
- Combine with router lazy-loading for scalable applications.
- Smartly prefetch resources to boost perceived speed.

🚀 Real-World Scenarios

1. Dashboards

Load analytics widgets and charts only when they're visible in the viewport. Perfect for admin panels with multiple heavy components that users may never scroll to.

```
@defer (on viewport) {
<analytics-widget
[data]="salesData"
[chartType]="'line'" />
<sales-chart [period]="'monthly'" />
<revenue-tracker />
} @placeholder {
<div class="skeleton-chart"></div>
} @loading (minimum 500ms) {
<spinner>Loading analytics...</spinner>
}
```

Reduces initial bundle by ~40-60KB

2. Third-party Widgets

Defer chat widgets, customer support tools, and analytics scripts until the browser is idle. These non-critical features shouldn't block your main content from loading.

```
@defer (on idle) {
<chat-widget
[apiKey]="chatConfig.key"
[position]="'bottom-right'" />
<intercom-messenger
[appId]="intercomId" />
<google-analytics-tracker />
} @placeholder {
<!-- No placeholder needed -->
}
```

Improves Time to Interactive (TTI)

3. Infrequent Dialogs

Load modal dialogs and pop-ups only when they're actually needed. Use conditional triggers to defer settings dialogs, confirmation modals, or feature tours that most users won't open.

```
@defer (when showSettingsModal) {
<settings-modal
[user]="currentUser"
(close)="showSettingsModal = false"
(save)="saveSettings($event)">
<account-tab />
<privacy-tab />
<notifications-tab />
</settings-modal>
}
```

Load on-demand, save ~30KB per dialog

4. Advertisement Banners

Defer ad banners until they enter the viewport. This prevents ads from blocking critical content and improves perceived performance. Add placeholders to prevent layout shift.

```
@defer (on viewport) {
<google-ad-banner
[adSlot]="'homepage-banner'"
[dimensions]="{ width: 728, height: 90
}" />
<native-ad-component
[placement]="'sidebar'" />
} @placeholder {
<div class="ad-placeholder"
style="height: 90px; background:
#f0f0f0;">
<span>Advertisement</span>
</div>
}
```

Prevents CLS, improves LCP score

5. Settings & Configuration Panels

Load advanced settings and configuration panels only when users interact with the trigger element. Great for collapsible sections with complex form controls and validation logic.

```
<button (click)="showAdvanced = true">
Show Advanced Settings
</button>
@defer (on interaction; when
showAdvanced) {
<advanced-settings-panel>
<api-configuration />
<webhook-settings />
<security-options />
<performance-tuning />
</advanced-settings-panel>
} @loading {
<loading-spinner />
}
```

85% of users never access these

6. Role-based & Premium Features

Conditionally load features based on user roles or subscription tiers. Why load premium features for free users? Use custom conditions to defer components that only specific user segments should see.

```
@defer (when user.isPremium &&
user.hasAccess('analytics')) {
<premium-features-panel>
<advanced-analytics-dashboard />
<ai-insights-widget />
<custom-reports-builder />
<export-tools />
</premium-features-panel>
} @placeholder {
<upgrade-prompt
message="Unlock premium analytics"
[ctaLink]="'/pricing'" />
}
```

Target loading by user segment

Start Optimizing Today!

Implement @defer to boost your Angular app performance

Happy coding! 🚀